Теоријска питања за одбрану пројекта

Елементи развоја софтвера

Питања служе за припрему усмене одбране пројекта. Одговор треба повезати са конкретним одлукама, кодом и тестовима сопственог пројекта.

У оквиру одбране пројекта студент одговара и на теоријска питања која обухватају теме обрађене на предмету и њихову примену на реализованом пројекту. Питања су груписана по тематским областима у наставку.

Захтеви листа планираних задатака Git и тимски развој

1. Објасните једну корисничку причу из вашег пројекта: која је улога, циљ и вредност коју доноси?
2. Како се пише корисничка прича по шаблону „Као [улога], желим/хоћу/могу [радњу], како бих [постигао циљ]“ и која је улога засебне чеклисте?
3. Покажите једну ставку листе планираних задатака и објасните кроз које је фазе прошла до статуса Завршено.
4. Зашто Гит историја треба да буде следљив запис развоја, а не само механизам за чување кода?
5. Шта чини квалитетну комит поруку и по чему се разликује од поруке попут „ажурирање“ или „исправка“?
6. Објасните један захтев за спајање измена из вашег репозиторијума: шта решава, шта мења, како је верификован?
7. Да ли сте имали Гит конфликт током развоја? Како сте одлучили које понашање треба да буде коначно?
8. Шта значи да је ставка листе планираних задатака заиста завршена (дефиниција завршености) у вашем пројекту?

Објектно-оријентисано програмирање чист код SOLID и чиста архитектура

1. Покажите пример интерфејса у вашем пројекту и објасните који уговор он дефинише.
2. Објасните принцип енкапсулације на конкретној класи - зашто стање није јавно изменљиво без контроле?
3. За једну класу у пројекту објасните која је њена једина одговорност (принцип једне одговорности).
4. Дајте пример где је принцип отворености и затворености примењен - како додајете ново понашање без измене постојећег кода?
5. Објасните принцип замене Барбаре Лисков на примеру из вашег кода - да ли се свака имплементација понаша у складу са уговором?
6. Шта је принцип раздвајања интерфејса и зашто широк интерфејс може бити проблем?
7. Објасните принцип инверзије зависности - да ли ваш пословни слој зависи од апстракција или од конкретних техничких детаља?
8. Нацртајте/опишите слојеве чисте архитектуре у вашем пројекту и смер зависности између њих.
9. Шта је корен композиције и где се у вашем пројекту налази регистрација конкретних зависности?
10. Који део вашег пројекта би највише патио када би доменски слој непосредно зависио од базе података?

Пословна логика и слој случајева употребе

1. Изаберите један случај употребе и објасните његов улаз, ток и могући исход (успех и неуспех).
2. Како ваш ентитет чува сопствене инваријанте и спречава неважеће стање?
3. Објасните разлику између пословног (очекиваног) неуспеха и техничке грешке у вашем систему.
4. Покажите пример где сте уместо логичке вредности, празне вредности или генеричког изузетка користили експлицитан резултат операције.
5. Које зависности ваш сервис случаја употребе добија кроз убризгавање зависности и зашто?
6. Објасните на ком нивоу се у вашем систему врши валидација улаза, а на ком пословних правила.

7. Која операција у вашем пројекту мора бити третирана као једна атомска пословна трансакција?
8. Коју архитектонску одлуку сте променили током развоја и зашто?

Тестабилни дизајн и аутоматизовано тестирање

1. Покажите један тест и објасните структуру „припреми, изврши, провери“ на њему.
2. Објасните разлику између позитивног, негативног и граничног тест сценарија у вашем пројекту.
3. Где и зашто сте користили Мок (или сличан алат) за изолацију зависности?
4. Зашто тестабилан дизајн (убризгавање зависности, уске границе) олакшава писање тестова без базе и мреже?
5. Који део вашег система је тестиран интеграционим, а не само јединичним тестом, и зашто?
6. Шта покривеност кода говори, а шта не говори о квалитету тестова?
7. Да ли сте неку грешку прво репродуковали тестом, а затим исправили? Опишите тај случај.
8. Шта је Гит ознака почетне верзије ручно развијеног језгра у вашем пројекту и зашто је тај тренутак важан?

Интеграција модула уговори и подаци

1. Које модуле има ваш пројекат и која је пословна граница сваког од њих?
2. Који модул поседује који податак, и како други модули том податку приступају?
3. Објасните уговор између два модула - шта клијент зна, а шта не зна о другој страни?
4. Када у вашем систему користите синхрони позив, а када пословни догађај? Зашто баш тако?
5. Дајте пример пословног догађаја из вашег система и објасните ко га објављује, а ко реагује на њега.
6. Како ваш систем решава конкурентан приступ истом ресурсу од стране два корисника или процеса?
7. Шта значи идемпотентност и где је у вашем пројекту потребна (нпр. поновљен захтев)?
8. Који интеграциони или уговорни тест штити границу између два ваша модула?

Контролисан развој уз подршку вештачке интелигенције

1. Која сте пројектна упутства за вештачку интелигенцију дефинисали и зашто баш та?
2. Покажите један запис из евиденције употребе ВИ и објасните која је одлука донета и како је верификована.
3. Коју сугестију ВИ алата сте прихватили, а коју одбацили? Образложите оба случаја.
4. Да ли сте направили сопствену процедуру за неки поновљиви задатак? Шта она ради?
5. Ако сте користили више агентских улога, која је одговорност и ограничење сваке од њих?
6. Зашто резултат ВИ алата сам по себи није довољан доказ да је промена исправна?
7. Када бисте у развоју вашег пројекта изабрали једноставан ток (један агент) уместо више улога?

MCP и повезивање са пројектом

1. Да ли ваш пројекат има MCP сервер? Које ресурсе или алате он излаже?
2. Објасните разлику између MCP ресурса и MCP алата на конкретном примеру из вашег пројекта.
3. Који стварни развојни сигнал (тест, преглед измена, статус) враћа неки од ваших MCP алата?
4. Како сте применили принцип најмањих привилегија при дефинисању дозвољених операција сервер-а?
5. Шта је убацивање злонамерних инструкција и како бисте (или јесте) заштитили агентски ток од непоузданог садржаја?
6. Који податак или тајну ваш MCP сервер намерно не излаже и зашто?

Извршиве провере заштитни механизми евалуације и завршна провера квалитета

1. Коју детерминистичку извршиву проверу има ваш пројекат и на који догађај се она окида?

2. Објасните разлику између инструкције (препоруке моделу) и извршиве провере (извршивог механизма).
3. Који заштитни механизам спречава ризичну операцију у вашем пројекту, и шта би се десило без њега?
4. Покажите један евалуациони сценарио (укључујући негативни) који проверава агентско понашање.
5. Ко је урадио колегијални преглед квалитета вашег пројекта и који је конкретан налаз или потврда забележена?
6. Који део вашег развојног тока мора бити гарантован кодом, а не само написан као инструкција?
7. Који је највећи преостали ризик или ограничење вашег пројекта и зашто сте га прихватили таквог?

Општа и интегративна питања

1. Проведите нас кроз цео пут једне функционалности: од захтева, преко кода и теста, до подршке вештачке интелигенције ако је коришћена.
2. Која одлука у пројекту је била најтежа и зашто сте је донели баш тако?
3. Шта бисте урадили другачије да сада поново крећете са пројектом?
4. Који део система не разумете у потпуности и зашто (искрено, не као трик питање)?
5. Објасните једну аналогију између класичног принципа софтверског инжењерства и његове примене у развоју уз вештачку интелигенцију (нпр. интерфејс - уговор алата, принцип једне одговорности - агентска улога).